

Spring Boot `ApplicationContext` Lifecycle

Chapter 2 — Spring Boot Internals · Pro Spring Boot 4 (4th ed) ·

`chapters/chapter-02-internals/customer`

Spring Boot 4.0.6

Spring Framework 7.0.7

Java 21 target / 25 runtime

Servlet + Tomcat 11

The `ApplicationContext` is the container Spring Boot builds and drives at startup. It is not “just dependency injection” — it **sequences** definition registration, instantiation, proxying, refresh and event publication. The example below traces each phase through the `customer` project and validates it against a live run.

The six phases

1 Bootstrap — `SpringApplication.run()`

`CustomerApplication.java:14-19`

The entry point uses a `SpringApplicationBuilder` (not the shortcut `SpringApplication.run`). It prints the banner (`bannerMode(CONSOLE)` → `banner.txt`), logs startup info, triggers **auto-configuration** (`@SpringBootApplication`), and — because a servlet API is present — creates an `AnnotationConfigServletWebServerApplicationContext` that also boots the embedded server.

2 Component scan & bean definition registration

@SpringBootApplication · @ConfigurationPropertiesScan · @EnableScheduling

Before any objects exist, the context reads metadata and registers **bean definitions** (blueprints):

- @SpringBootApplication scans com.apress.crm.customer for @Component / @Configuration / @RestController.
- @ConfigurationPropertiesScan discovers CrmProperties (@ConfigurationProperties(prefix = "crm")) and binds application.properties.
- @EnableScheduling registers infrastructure that later scans for @Scheduled methods.

3 Bean instantiation + dependency injection

constructor injection throughout

Definitions become live singletons and the constructor dependency graph resolves: CustomerRepository → CrmProperties + auto-configured MessageSource → CustomerController (CustomerController.java:19) and CustomerStatsTask. CustomerConfiguration is @Profile("!prod"), so its beans register only when prod is inactive.

4 AOP proxy creation

ControllerLoggingAspect.java:10-24

@Aspect + @Around("execution(* ...CustomerController.*(..))") makes the context wrap CustomerController in a **CGLIB proxy**. What other beans inject is the proxy, so every call gets latency logging transparently. Spring Boot 4 defaults to CGLIB/byte-buddy proxies.

5 Refresh completes → lifecycle events

CustomerConfiguration.java:17-26

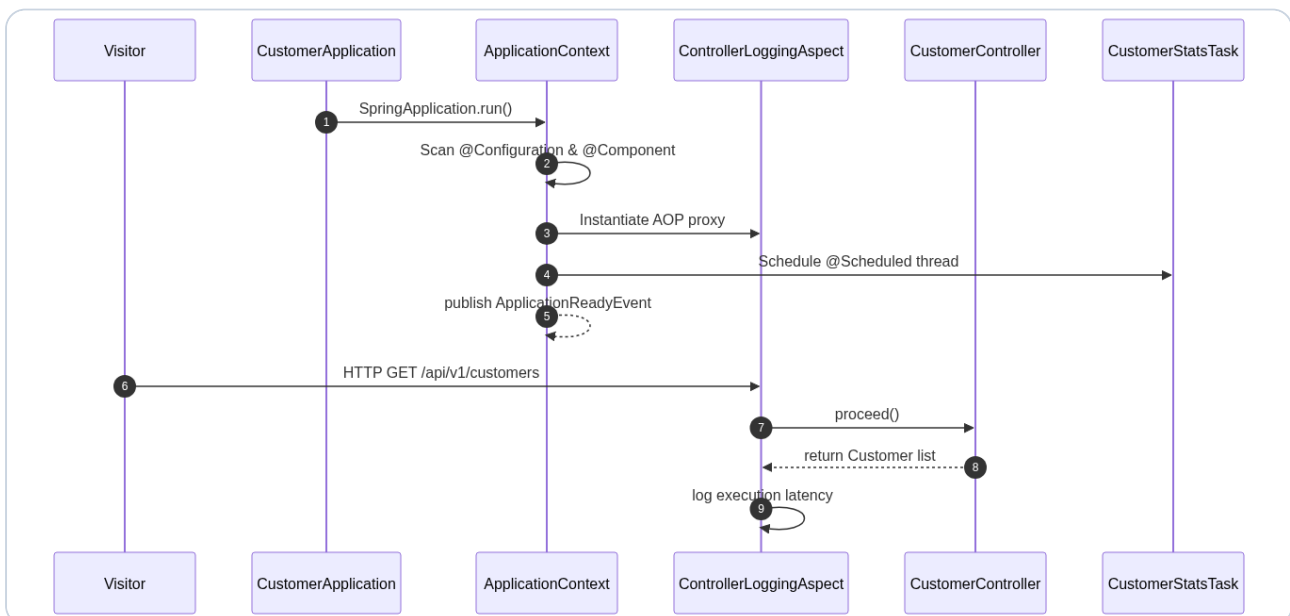
The server is started and singletons fully initialized, so the context publishes `ApplicationReadyEvent`. The `ApplicationListener<ApplicationReadyEvent>` bean seeds three sample customers — the canonical “run after the whole context is up” pattern, versus doing work in a constructor.

6 Steady state — post-startup runtime

@Scheduled + HTTP dispatch

Once ready, the long-lived context hosts `@Scheduled(fixedRate = 60000)` on `CustomerStatsTask` (runs on the scheduling thread) and serves HTTP through the proxied `CustomerController`.

Sequence diagram



Rendered from a Mermaid `sequenceDiagram` and embedded as a self-contained image (works offline, no scripts, and prints in the PDF).

Flow at a glance

```
main() → SpringApplicationBuilder.run()
  → create AnnotationConfigServletWebServerApplicationContext
  → auto-configuration + component scan → register BEAN DEFINITIONS
  → instantiate singletons + resolve constructor DEPENDENCY INJECTION
  → apply AOP → wrap CustomerController in CGLIB proxy
  → finish refresh → start embedded Tomcat server
  → publish ApplicationReadyEvent → CustomerConfiguration seeds data
  → steady state: @Scheduled tasks + serving requests
```

Live-run evidence

The following was captured from an actual `mvn spring-boot:run` of the project.

Phases 1-3 — bootstrap, scan, refresh

```
(::Customer CRM::. Version 1.0 / Using Spring Boot 4.0.6)      ← banner (phase 1)
Starting CustomerApplication using Java 25.0.4 with PID 27007
Running with Spring Boot v4.0.6, Spring v7.0.7
No active profile set, falling back to 1 default profile: "default" ← !prod config WILL register
Tomcat initialized with port 8080 (http)
Root WebApplicationContext: initialization completed in 4558 ms  ← phases 2-3
Tomcat started on port 8080 (http) with context path '/'
Started CustomerApplication in 8.418 seconds
```

Phase 5 — `ApplicationReadyEvent` (fires after “Started”)

```
DEBUG CustomerRepository : Saving customer: John
DEBUG CustomerRepository : Saving customer: Jane
DEBUG CustomerRepository : Saving customer: Jack
Customer[id=22d525bd-..., name=Jane, email=jane@example.com, ...] ← findAll().forEach(out::println)
Customer[id=068c4bca-..., name=Jack, email=jack@example.com, ...]
Customer[id=e912d055-..., name=John, email=john@example.com, ...]
```

Phase 6 — scheduled task on its own thread

```
INFO [ scheduling-1] CustomerStatsTask : CRM Stats: Total customers currently in system: 3
```

Phase 4 — AOP proxy + config binding + i18n (request-time)

```
GET /api/v1/customers → INFO [nio-8080-exec-1] ControllerLoggingAspect :  
    Iterable CustomerController.findAll() executed in 1ms ← proxy on the request thread  
  
GET /api/v1/customers/welcome → "Welcome to the Pro Spring Boot 4 CRM!" ← @ConfigurationProperties  
GET /api/v1/customers/greet/Carlos (Accept-Language: es)  
    → "Hola, Carlos! Bienvenido a nuestro CRM." ← MessageSource i18n
```

Feature → lifecycle stage mapping

Chapter feature	Symbol	Lifecycle stage
Entry point / banner / profiles	<code>SpringApplicationBuilder</code>	1 · Bootstrap
Bean discovery	<code>@SpringBootApplication</code> component scan	2 · Definition registration
Typed external config	<code>@ConfigurationProperties(prefix="crm")</code> on <code>CrmProperties</code>	2 → 3 · Binding
Collaborators	constructor injection	3 · Instantiation + DI
Profile-conditional beans	<code>@Profile("!prod")</code>	2 · Registration (filtered)
Cross-cutting logging	<code>@Aspect @Around(...Controller.*(..))</code>	4 · AOP proxying
Post-startup work	<code>ApplicationListener<ApplicationReadyEvent></code>	5 · Event publication
Background jobs	<code>@EnableScheduling</code> + <code>@Scheduled</code>	6 · Post-refresh runtime

Engineering insights.

- Spring Boot 4 initializes proxy beans with **CGLIB / byte-buddy** by default.
- Prefer **constructor-bound** `@ConfigurationProperties` over `@Value` for strong typing, testability, and IDE autocomplete.
- Seed or verify state in an `ApplicationReadyEvent` listener, not a constructor — the context (and server) is not fully ready inside a constructor.

Generated from `chapters/chapter-02-internals/` · source at

`.../customer/src/main/java/com/apress/crm/customer/`. Fully self-contained (no external scripts).

Build & run: `cd chapters/chapter-02-internals/customer && mvn spring-boot:run`